

Source code for nltk.metrics.distance

```
# Natural Language Toolkit: Distance Metrics
#
# Copyright (C) 2001-2017 NLTK Project
# Author: Edward Loper <edloper@gmail.com>
#         Steven Bird <stevenbird1@gmail.com>
#         Tom Lippincott <tom@cs.columbia.edu>
# URL: <http://nltk.org/>
# For license information, see LICENSE.TXT
#

"""
Distance Metrics.

Compute the distance between two items (usually strings).
As metrics, they must satisfy the following three requirements:

1.  $d(a, a) = 0$ 
2.  $d(a, b) \geq 0$ 
3.  $d(a, c) \leq d(a, b) + d(b, c)$ 
"""

from __future__ import print_function
from __future__ import division

def _edit_dist_init(len1, len2):
    lev = []
    for i in range(len1):
        lev.append([0] * len2) # initialize 2D array to zero
    for i in range(len1):
        lev[i][0] = i # column 0: 0,1,2,3,4,...
    for j in range(len2):
        lev[0][j] = j # row 0: 0,1,2,3,4,...
    return lev

def _edit_dist_step(lev, i, j, s1, s2, substitution_cost=1, transpositions=False):
    c1 = s1[i - 1]
    c2 = s2[j - 1]

    # skipping a character in s1
    a = lev[i - 1][j] + 1
    # skipping a character in s2
    b = lev[i][j - 1] + 1
    # substitution
    c = lev[i - 1][j - 1] + (substitution_cost if c1 != c2 else 0)

    # transposition
    d = c + 1 # never picked by default
    if transpositions and i > 1 and j > 1:
        if s1[i - 2] == c2 and s2[j - 2] == c1:
            d = lev[i - 2][j - 2] + 1

    # pick the cheapest
    lev[i][j] = min(a, b, c, d)

def edit_distance(s1, s2, substitution_cost=1, transpositions=False):
    """
    Calculate the Levenshtein edit-distance between two strings.
    The edit distance is the number of characters that need to be
    substituted, inserted, or deleted, to transform s1 into s2. For
    example, transforming "rain" to "shine" requires three steps,

```

[\[docs\]](#)

TABLE OF CONTENTS

NLTK News
Installing NLTK
Installing NLTK Data
Contribute to NLTK
FAQ
Wiki
API
HOWTO

SEARCH

Go

consisting of two substitutions and one insertion:

"rain" -> "sain" -> "shin" -> "shine". These operations could have been done in other orders, but at least three steps are needed.

Allows specifying the cost of substitution edits (e.g., "a" -> "b"), because sometimes it makes sense to assign greater penalties to substitutions.

This also optionally allows transposition edits (e.g., "ab" -> "ba"), though this is disabled by default.

```
:param s1, s2: The strings to be analysed
:param transpositions: Whether to allow transposition edits
:type s1: str
:type s2: str
:type substitution_cost: int
:type transpositions: bool
:rtype int
"""

# set up a 2-D array
len1 = len(s1)
len2 = len(s2)
lev = _edit_dist_init(len1 + 1, len2 + 1)

# iterate over the array
for i in range(len1):
    for j in range(len2):
        _edit_dist_step(lev, i + 1, j + 1, s1, s2,
                        substitution_cost=substitution_cost, transpositions=transpositions)
return lev[len1][len2]
```

def `binary_distance(label1, label2):` [\[docs\]](#)

"""Simple equality test.

0.0 if the labels are identical, 1.0 if they are different.

```
>>> from nltk.metrics import binary_distance
>>> binary_distance(1,1)
0.0
```

```
>>> binary_distance(1,3)
1.0
"""
```

```
return 0.0 if label1 == label2 else 1.0
```

def `jaccard_distance(label1, label2):` [\[docs\]](#)

"""Distance metric comparing set-similarity.

"""

```
return (len(label1.union(label2)) - len(label1.intersection(label2)))/len(label1.union(label2))
```

def `masi_distance(label1, label2):` [\[docs\]](#)

"""Distance metric that takes into account partial agreement when multiple labels are assigned.

```
>>> from nltk.metrics import masi_distance
>>> masi_distance(set([1, 2]), set([1, 2, 3, 4]))
0.335
```

*Passonneau 2006, Measuring Agreement on Set-Valued Items (MASI)
for Semantic and Pragmatic Annotation.*

```
len_intersection = len(label1.intersection(label2))
len_union = len(label1.union(label2))
```

```

len_label1 = len(label1)
len_label2 = len(label2)
if len_label1 == len_label2 and len_label1 == len_intersection:
    m = 1
elif len_intersection == min(len_label1, len_label2):
    m = 0.67
elif len_intersection > 0:
    m = 0.33
else:
    m = 0

return (1 - (len_intersection / float(len_union))) * m

```

def interval_distance(label1,label2): [\[docs\]](#)

"""Krippendorff's interval distance metric

>>> from nltk.metrics import interval_distance
>>> interval_distance(1,10)
81

Krippendorff 1980, Content Analysis: An Introduction to its Methodology
"""

```

try:
    return pow(label1 - label2, 2)
#     return pow(list(label1)[0]-list(label2)[0],2)
except:
    print("non-numeric labels not supported with interval distance")

```

def presence(label): [\[docs\]](#)

"""Higher-order function to test presence of a given label
"""

```

return lambda x, y: 1.0 * ((label in x) == (label in y))

```

def fractional_presence(label): [\[docs\]](#)

```

return lambda x, y:\
    abs(((1.0 / len(x)) - (1.0 / len(y)))) * (label in x and label in y) \
    or 0.0 * (label not in x and label not in y) \
    or abs((1.0 / len(x))) * (label in x and label not in y) \
    or ((1.0 / len(y))) * (label not in x and label in y)

```

def custom_distance(file): [\[docs\]](#)

```

data = {}
with open(file, 'r') as infile:
    for l in infile:
        labelA, labelB, dist = l.strip().split("\t")
        labelA = frozenset([labelA])
        labelB = frozenset([labelB])
        data[frozenset([labelA,labelB])] = float(dist)
return lambda x,y:data[frozenset([x,y])]

```

def demo(): [\[docs\]](#)

```

edit_distance_examples = [
    ("rain", "shine"), ("abcdef", "acbdef"), ("language", "lnaguaeg"),
    ("language", "lnaugage"), ("language", "lngauage")]
for s1, s2 in edit_distance_examples:
    print("Edit distance between '%s' and '%s': " % (s1, s2), edit_distance(s1, s2))
for s1, s2 in edit_distance_examples:
    print("Edit distance with transpositions between '%s' and '%s': " % (s1, s2), edit_distance(s1, s2, transpositions=True))

```

```
s1 = set([1, 2, 3, 4])
s2 = set([3, 4, 5])
print("s1:", s1)
print("s2:", s2)
print("Binary distance:", binary_distance(s1, s2))
print("Jaccard distance:", jaccard_distance(s1, s2))
print("MASI distance:", masi_distance(s1, s2))

if __name__ == '__main__':
    demo()
```

[MODULES](#) | [INDEX](#)

© Copyright 2017, NLTK Project. Last updated on May 21, 2017. Created using [Sphinx](#) :